Data Structures

RECITATION 1 - INTRODUCTION

Administration

Piazza (link)

- Course materials (lecture notes, homework assignments, and solutions)
- Q&A forum – use English
- **All course-related questions/issues should be posted on Piazza!**
- Only use private posts (viewed only by staff) for private matters
- You can post anonymously
- You are encouraged to help each other

Moodle

- Only used for submitting homework, grades, and access to Panopto.

Homework Submission

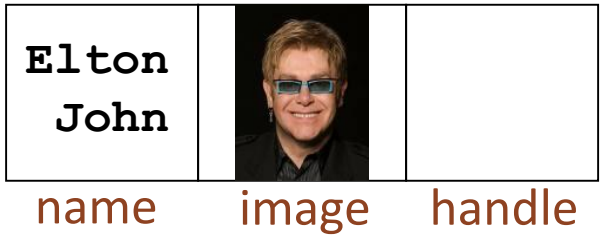
- Usually posted on Friday, for submission the following Friday.
- All homework assignments must be submitted by the posted deadline.

Today

- ADT vs. Implementation
- Math Essentials for Data Structures
  - Proof by Induction
  - Arithmetic and Geometric Series
  - Logarithms and Exponents
  - Pseusocode

A List Element: an Object Which Contains both data and a Handle to another List Element.

Example List Element x:



To access the fields we use

Example: x.name
x.image
x.next

OR

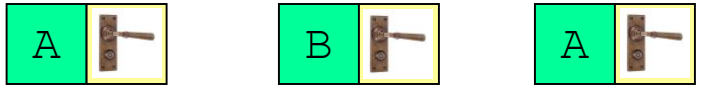
name[x]
image[x]
next[x]

ADT: Linked List

Objects: A finite sequence of List Elements

Operations:

- Create list
- Get the first List Element
- Insert new List Element as first
- Delete first List Element

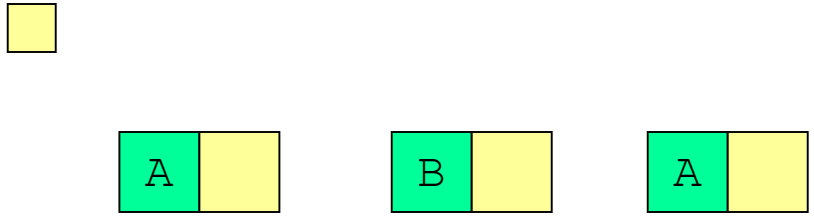


Given a List Element:

- Get data
- Set data
- Get next List Element
- Set next List Element

Data Structure: Linked List

special handle: head



(trivial) implementation of the operations:

for example:

Get the first List Element of linked list L:

return $head[L]$

Get data of List Element x:

return $data[x]$

Pseudo Code

head[L]

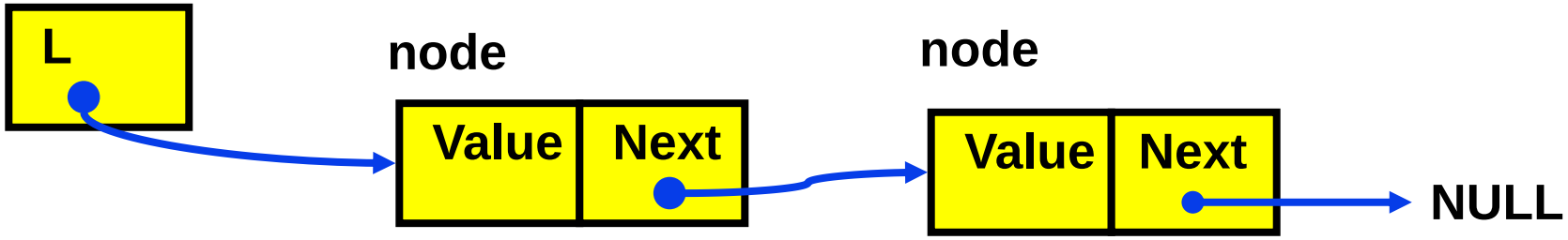
next[x]

data[x]

Linked List Implementation #1 by (actual) Node Objects

Basic Idea: A node object has

- Value: For each node allocate memory
- Next: Keep a handle to the memory location of the following Node (ie, pointer, reference)



Linked List Implementation #2 by “Naïve” array – like Sequence

Complexity of node insertion

- Best case: $O(1)$
- Worst case: $O(n)$
- Average case: $O(n)$

Complexity of node deletion

- Best case: $O(1)$
- Worst case: $O(n)$
- Average case: $O(n)$

Linked List Implementation #3
by Data and handles stored in Arrays

Why?

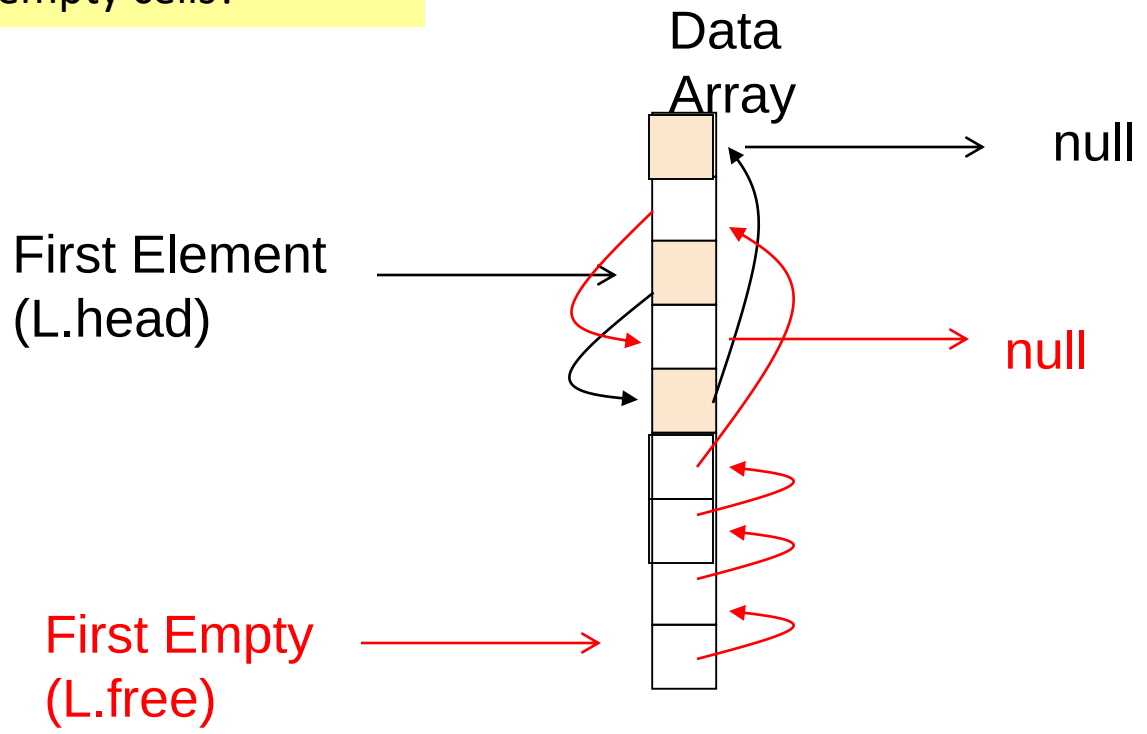
- Using actual pointers is difficult or not supported in some programming languages
  (Fortran, Basic, etc…)
- Sometimes more efficient
  (e.g., when number of elements is known)

How?

- An array of the data
- An integer array of indexing - the handles

Idea

Where are the empty cells?

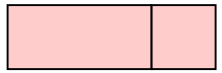


Idea

Abstract view

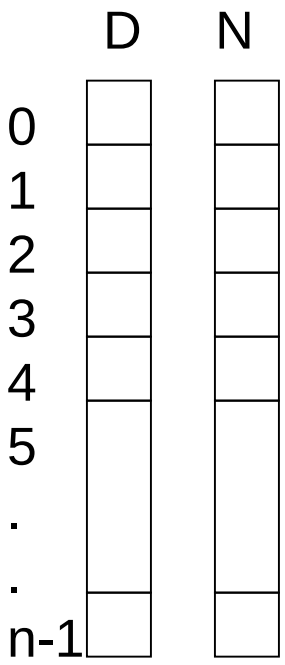
n nodes

data next



value : some data
next : handle to next node

Implementation by Arrays



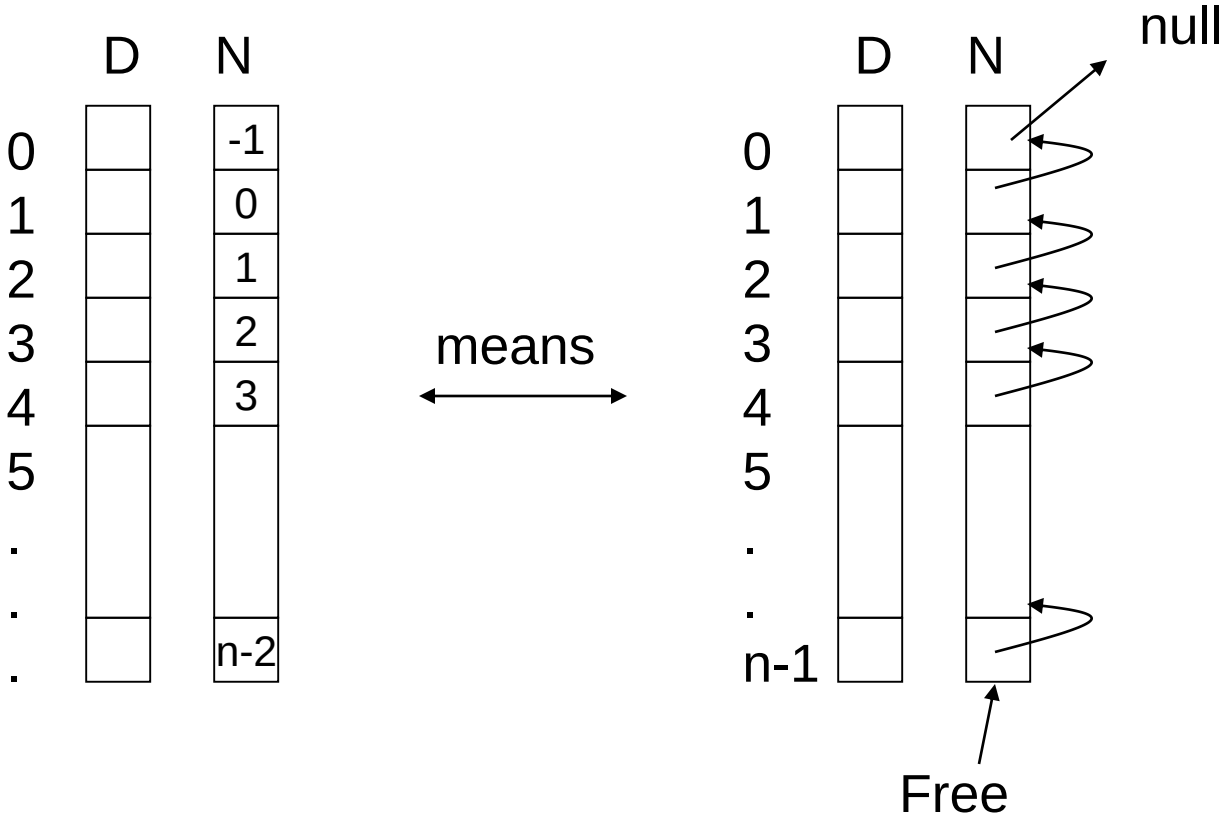
L is a structure:

- L.D[ ] : basic type array
- L.N[ ] : integer array
- L.n: array size
- L.head: index of the first element in the data list
- L.free: index of the first element in the free list

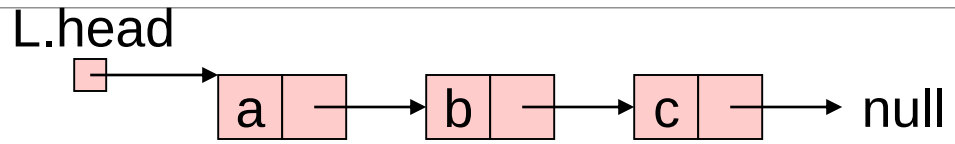
- Pointer is an integer
- null is -1
- p.data is L.D[p]
- p.next is L.N[p]

Initialization

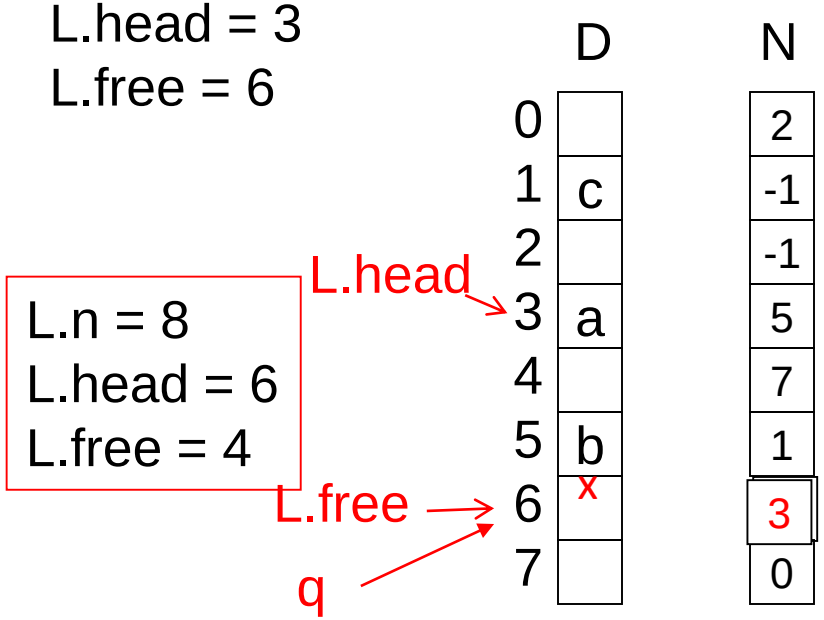
L.Data
L.n
L.free = n-1
L.head=null



Example of Use



L.n = 8
L.head = 3
L.free = 6



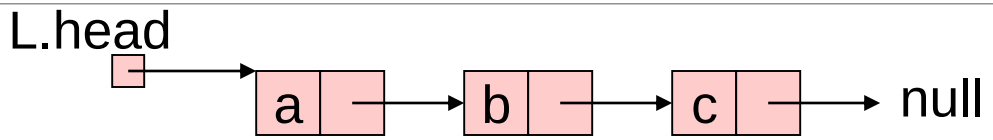
L.n = 8
L.head = 6
L.free = 4

L.D: data array
L.N: indexes array

```
InsertFront (L, x)
    if (L.free ≠ -1) then
        q ← L.free
    else
        return "overflow";
    L.D[q] ← x;
    L.free ← L.N[L.free];
    L.N[q] ← L.head;
    L.head ← q;
```

Example of Use

Double the array size



L.n = 8
L.head = 3
L.free = 6

	D	N
0		2
1	c	-1
2		-1
3	a	5
4		7
5	b	1
6		4
7		0

```
InsertFront (L, x)
    if (L.free ≠ -1) then
        q ← L.free
    else
        return "overflow",
    L.free ← L.N[q];
    L.D[q] ← x;
    L.N[q] ← L.head;
    L.head ← q;
```

L.D: data array
L.N: indexes array

DeleteFront

```
DeleteFront (L)
    if L.head = -1 then
        return "underflow"
    else
      q ← L.head;
      L.head ← L.N[q];
      L.N[q] ← L.free;
      L.free ← q;
```

L.head = 3
L.free = 6

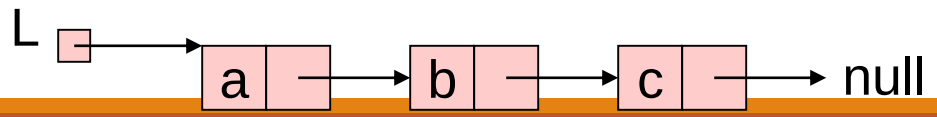
	D	N
0		2
1	c	-1
2		-1
3	a	5
4		7
5	b	1
6		4
7		0

before

L.head=5
L.free = 3

D	N
	2
c	-1
	-1
	6
	7
b	1
	4
	0

after



Proofs By Induction - Overview

Claim: $S(n)$ is true for all $n \geq c$

Base case: Show $S(n)$ is true for $n = c$ (usually for c = 0 or 1)

Inductive hypothesis: Assume $S(k)$ is true for all $k < n$

Inductive step: Show that $S(n)$ is then true for n

Induction proves a statement $S(n)$ holds for all $n \geq c$ by building on smaller cases.

Note that many times we only assume the claim holds just for $k = n - 1$.

Example: prove that $\sum_{i=1}^{n} i = 1 + 2 + \cdots + n - 1 + n = \frac{\boldsymbol{n(n+1)}}{\mathbf{2}}$ for any $n \geq 1$

Base case: $n = 1$. Clearly, $\sum_{i=1}^{1} i = 1 = \frac{1(1+1)}{2} = 1.$

Induction hypothesis: Assume the claim holds for $n = k$: $\sum_{i=1}^{k} i = \frac{k(k+1)}{2}$

Induction step: We will show correctness for $n = k + 1$.
the sum up to $k + 1$ may be written as: $\sum_{i=1}^{k+1} i = \sum_{i=1}^{k} i + (k + 1)$

Using the induction assumption, it becomes:

$$\sum_{i=1}^{k+1} i = \sum_{i=1}^{k} i + (k + 1) = \frac{k(k+1)}{2} + (k + 1) = \frac{k(k+1)+2(k+1)}{2} =$$
$$= \frac{\boldsymbol{(k+1)(k+2)}}{\mathbf{2}}$$

This matches the claim for $n = k + 1$. Thus, by induction, the formula is true for all $n \geq 1$.

Induction - All horses are the same color

The proof consists of three steps:

The **base case**: If there is only one horse in the group, all horses in that group are clearly the same color.

The **inductive hypothesis**: Assume that groups of n horses always have the same color.

The **inductive step:** Let us consider a group consisting of $n + 1$ horses.

Induction - All horses are the same color

The **inductive Step:** Let us consider a group consisting of $n + 1$ horses.

1. **Exclude the last horse**, leaving $\boldsymbol{n}$ **horses**.

• By the induction hypothesis, these $\boldsymbol{n}$ **horses** must all have the same color.

2. Now, **exclude the first horse**, again leaving another set of $\boldsymbol{n}$ **horses**.

• Again, by the induction hypothesis, this second set of $\boldsymbol{n}$ **horses** must all have the same color.

3. Therefore, the first horse in the group is of the same color as the horses in the middle, who in turn are of the same color as the last horse. Hence, the first horse, middle horses, and last horse are all of the same color, and we have proven that: If n horses have the same color, then $n + 1$ horses will also have same color.

Why the Proof Actually Fails?

For this reasoning to work properly, there must be an **overlap** between these two groups of n horses, ensuring a horse common to both groups that links the colors.

This induction argument fails specifically at the transition from $\boldsymbol{n = 1}$ to $\boldsymbol{n = 2}$:

- For $\boldsymbol{n = 2}$, the two subsets created (when removing the first and last horse) each contain just **one horse**, with **no overlap**. Thus, we cannot conclude that both horses share the same color.

- Induction requires the inductive step to hold **for all** transitions from k to $k + 1$, **but here it fails** at $n = 2$ because the sets $\{h_1\}$ and $\{h2_2\}$ don't overlap.

WHAT SEEMS TO BE THE OFFICER PROBLEM?
quickmeme.com

Math Refresher

Let’s go over some topics you should have learned in previous classes and discuss how they’ll be helpful in Data Structures.

If you don’t remember/these topics are new to you– please review the topics!

Arithmetic Sequence

An **arithmetic sequence** is a sequence of numbers such that the difference between consecutive terms is constant.

For instance, the sequence 5, 7, 9, 11, 13, 15, . . . is an arithmetic progression with a *common difference* of 2.

Arithmetic Sequence

$\boldsymbol{a_1}$ - Initial term of an arithmetic sequence.

$\boldsymbol{d}$ - common difference of successive members.

$\boldsymbol{a_n} = a_{n-1} + d = a_1 + (n-1)d$

And in general:

$a_n = a_m + (n-m)d$

Arithmetic Sequence - Sum

$$S_n = \frac{n}{2}(a_1 + a_n)$$

Which is also equal to

$$S_n = \frac{n}{2}[2a_1 + (n-1)d]$$

For example,

$$2 + 5 + 8 + 11 + 14 = \frac{5(2+14)}{2} = 40$$

Geometric Sequence

A **geometric sequence** is a sequence of numbers where each term after the first is the result of multiplying the previous term by a fixed, non-zero number called the *common ratio*.

For example, the sequence 2, 6, 18, 54, ... is a geometric progression with a common ratio of 3.

Similarly, 10, 5, 2.5, 1.25, ... is a geometric sequence with a common ratio of $\frac{1}{2}$.

Geometric Sequence

Examples of a geometric sequence are powers r^k of a fixed number r, such as 2^k and 3^k. The general form of a geometric sequence is a, ar, ar^2, ar^3, ar^4 …

$\boldsymbol{a}$ - Initial term of a geometric sequence.

$\boldsymbol{r}$ – common ratio of successive members.

$$\boldsymbol{a_n} = ar^{n-1}$$

$$\boldsymbol{a_n} = a_{n-1} \cdot r$$

Geometric Sequence - Sum

$S_n = \frac{a(1-r^n)}{1-r}$ or $S_n = \frac{a(r^n-1)}{r-1}$ (preferred when $r > 1$)

For example,

$$2 + 10 + 50 + 250 = 2 + 2 \cdot 5 + 2 \cdot 5^2 + 2 \cdot 5^3 = \frac{2(1-5^4)}{1-5} = 312$$

Infinite Geometric Sequence

When $0 < r < 1$ and n is large, then

$$S_n = \frac{a(1-r^n)}{1-r} \approx \frac{a}{1-r}$$

For example,

$$\frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \cdots \approx \frac{\frac{1}{2}}{1-\frac{1}{2}} = 1$$

Exponents – useful identities

- $a^0 = 1$
- $a^1 = a$
- $a^{-1} = \frac{1}{a}$
- $a^{m+n} = a^m \cdot a^n$
- $(a^m)^n = a^{mn} = (a^n)^m$
- $(b \cdot c)^n = b^n \cdot c^n$

Logarithms

$\boldsymbol{log_b\, x}$ is the unique real number y such that $b^y = x$.

For example, $\log_2 64 = 6$, as 64 = 2^6 .

- For all real a > 0, b > 0, c > 0, and n, and logarithm bases are not 1:

$$
\begin{aligned}
a &= b^{\log_b a}, \\
\log_c(ab) &= \log_c a + \log_c b, \\
\log_b a^n &= n \log_b a, \\
\log_b a &= \frac{\log_c a}{\log_c b},
\end{aligned}
\qquad
\begin{aligned}
\log_b(1/a) &= -\log_b a, \\
\log_b a &= \frac{1}{\log_a b}, \\
a^{\log_b c} &= c^{\log_b a}.
\end{aligned}
$$

[Logarithm, from Wikipedia]

Example

Claim: $\boldsymbol{\log_a bc = \log_a b + \log_a c}$

By the definition of the log function:

$b = a^{\log_a b}, \quad \mathrm{c} = a^{\log_a c}, \quad bc = a^{\log_a bc}$ (exponent property)

Hence,

$a^{\boldsymbol{log_a bc}} = bc = a^{\log_a b} \cdot a^{\log_a c} = a^{\boldsymbol{log_a b + log_a c}}$

Logarithms – base change

The logarithm $\log_b x$ can be computed from the logarithms of x and b with respect to an arbitrary base k using the following formula:

$$\log_b x = \frac{\log_k x}{\log_k b}.$$

Example:

$$\log_b x = \frac{\log_{10} x}{\log_{10} b} = \frac{\log_e x}{\log_e b}.$$

Changing the base changes the value by a constant factor.

Floor and Ceiling Functions

$\lfloor x \rfloor$ – Floor function: the largest integer $\leq X$

$\lfloor -2.7 \rfloor = -3$ $\lfloor 2.7 \rfloor = 2$ $\lfloor 2 \rfloor = 2$

$\lceil x \rceil$ – Ceiling function: the smallest integer $\geq X$

$\lceil -2.3 \rceil = -2$ $\lceil 2.3 \rceil = 3$ $\lceil 2 \rceil = 2$

Induction – Yet Another Example

Define the following sequence: $a_1 = 0$ and for every $n \geq 2$, $a_n = a_{\lfloor \frac{n}{2} \rfloor} + 1$.

Prove that if $n = 2^k$, for some positive integer k, then $a_n = \log_2(n) = k$.

We prove by induction on k:

- **Base:** When $k = 1$, $n = 2^1 = 2$. By definition, $a_2 = a_1 + 1 = 0 + 1 = 1$. Indeed, $\log_2 2 = 1$.
- **Inductive Hypothesis:** For some $k \geq 1$, $n = 2^k$. By definition, $a_n = a_{2^k} = \log_2(2^k) = k$.
- **Inductive Step:** Prove for $k + 1$, $n = 2^{k+1}$. By definition, $a_{2^{k+1}} = a_{\lfloor \frac{2^{k+1}}{2} \rfloor} + 1 = a_{2^k} + 1$.

Apply induction hypothesis: $a_{2^k} + 1 = k + 1 = \log_2(2^{k+1})$

And clearly, $\log_2(2^{k+1}) = k + 1$. Thus, the induction step holds clearly.

Pseudocode Exercise – Find Max

Write pseudocode for a function "Find-Max". The function receives a Linked List of numbers as input, and must return the maximum number in the list.

```
Find-Max(list)
  curr = list.head
  curr_max = -infinity
  while curr != null
    curr_max = Maximum(curr_max, curr.data)
    curr = curr.next
  return curr_max
```

Checklist:

- Is the pseudocode unambiguous?
- Can a person understand it regardless of the programming language(s) they prefer?
- Could someone turn this into a program?

If "Yes" to all, it's good pseudocode

Tip: don't stress about the "syntax" of pseudocode. So long as it's clear, the syntax is correct